Android 屏幕翻译

技术栈方案

采用 Kotlin + Jetpack Compose + Android 原生截图 / 悬浮窗 + ONNX Runtime Android + PP-OCRv6 small + 可配置的联网翻译服务。

这是待验证的方案：尚未建立 Android 工程，也没有真机结果。先验证跨应用取图是否顺手，以及现有模型在手机上的完整识别速度和准确率。

版本 2026-10-27　状态：方案，未真机验证　配套：讲解书、开发计划

目 录

1 推荐结论
2 技术选择与对应价值
3 Android 的实际操作约束
4 PP-OCRv6 small 能否沿用
5 一次使用的数据流
6 翻译、联网与数据
7 通用版兼容与发布基线
8 目前证据与待验证项

1 推荐结论

> **本方案对应需求：**通用 Android 版、轻便流畅、本地高精度 OCR、退到后台后使用侧边悬浮键、点击后在截图上手动框选。首版默认支持英日译中，同时验证中英日 OCR；不把“通用版”理解为所有手机和所有应用均可用。

采用 **Kotlin + Jetpack Compose + Android 原生截图 / 悬浮窗 + ONNX Runtime Android + PP-OCRv6 small + 可配置的联网翻译服务**。

沿用 STOR 的“截图 → 框选 → 本地识别 → 可修改原文 → 主动点击翻译”体验。Android 单独建立工程，复用模型、配置、提示词和验证样本；桌面窗口、截图、托盘与 Python 子进程不直接迁移。

先验证两个最大未知数：跨应用取图是否顺手，以及现有模型在手机上的完整识别速度和准确率。通过后再打磨完整界面。

未知数 01

跨应用取图是否顺手

一次授权会话内，能否稳定拿到干净的新帧，并让浮键、旧结果不进入截图。

未知数 02

手机端识别是否达标

现有模型在手机上完整跑通的精度和速度是否达到可用。

本章要点

- 路线是 Kotlin 原生 + ORT + PP-OCRv6 small，先复用现有模型与样本。
- 先验证取图和手机端识别，再打磨完整界面。

2 技术选择与对应价值

首版技术选择
部分 **开发语言** 首版选择 Kotlin 对用户的价值 / 实现边界 直接调用 Android 系统能力，减少截图和后台服务的跨框架衔接。
部分 **主应用界面** 首版选择 Jetpack Compose + Material 3 对用户的价值 / 实现边界 快速实现首页、设置、授权说明及深浅主题；流畅程度仍靠实测。
部分 **侧边按钮、截图框选、结果卡片** 首版选择 WindowManager + TYPE_APPLICATION_OVERLAY；简单原生 View，框选使用 Canvas 对用户的价值 / 实现边界 不依赖切回首页；框选层只绘制冻结图、遮罩和选框。
部分 **系统截图** 首版选择 MediaProjection + 一个 VirtualDisplay + ImageReader 对用户的价值 / 实现边界 在用户授权的会话中按需取得当前画面；不录音、不保存录像。

部分

后台会话

首版选择

mediaProjection 类型的前台服务

对用户的价值 / 实现边界

为投屏会话提供明确生命周期和停止入口；不能保证永不被系统结束。

部分

OCR 推理

首版选择

官方完整 onnxruntime-android，先用 CPU

对用户的价值 / 实现边界

先确保现有 ONNX 模型和完整流水线正确，再按证据优化。

部分

OCR 模型

首版选择

PP-OCRv6_small_det + PP-OCRv6_small_rec

对用户的价值 / 实现边界

优先复用当前 STOR 的两个模型，实际精度和延迟须 Android 验证。

部分

图像处理

首版选择

OpenCV Android，配合对齐后的 DB 后处理和 CTC 解码

对用户的价值 / 实现边界

完成检测框提取、裁剪、识别解码；ONNX 推理本身不包含这些工作。

部分

异步执行

首版选择

Kotlin Coroutines + StateFlow

对用户的价值 / 实现边界

截图转换、OCR、网络请求不阻塞手指拖动和界面反馈。

部分

翻译请求

首版选择

OkHttp + kotlinx.serialization；支持流式结果

对用户的价值 / 实现边界

接入现有兼容 Chat Completions 的服务，先显示已返回的译文。

部分

普通设置

首版选择

DataStore

对用户的价值 / 实现边界

保存目标语言、服务地址、模型标识、悬浮位置等少量配置。

部分

API Key

首版选择

Android Keystore 生成加密密钥，API Key 密文保存在应用私有目录

对用户的价值 / 实现边界

Keystore 保存加密密钥，不是把 API Key 文本直接塞进 Keystore。

部分

缓存

首版选择

有上限的会话内存缓存

对用户的价值 / 实现边界

重复文字少请求；首版不引入数据库和历史记录。

部分

体验验证

首版选择

真机、Android Studio Profiler / Perfetto，关键路径 Macrobenchmark

对用户的价值 / 实现边界

分别量化交互、OCR、内存与耗电，避免仅凭模拟器感觉判断。

> 界面层不要把全分辨率 Bitmap 放进可反复序列化的状态；捕获资源由会话持有，界面仅取得当前显示所需引用。首版单 app 模块，按职责分包即可，不建立多模块、插件式 OCR 框架或通用

> 任务调度系统。

为什么选择原生 Android

路线	本项目适合度	取舍
Kotlin 原生	推荐	主要难点都是 Android 专属能力，平台交互集中、排查路径短。
Flutter + Kotlin 原生插件	可做，当前不优先	Flutter 能完成主界面，但截图、悬浮窗和后台会话仍需要原生实现；未来明确要共享移动界面时再评估。
延用 Tauri / React 外壳	当前不优先	可分享部分界面思路，但 Windows 截图与 OCR 子进程不能因此复用；本项目没有必要为设置页承担额外桥接工作。

原生框架不自动保证“丝滑”；真正决定体验的是取帧时序、选区手势、主线程工作量、OCR 耗时和网络反馈。

易错点

选型表不是性能证明，**原生框架也不自动保证丝滑**。流畅度要靠真机时序与主线程工作量验证。

本章要点

- 每项技术都对应一个用户价值或实现边界，不只是“用了什么”。
- 原生框架不自动保证流畅；取帧、手势、主线程、OCR 和网络才是体验变量。

3 Android 的实际操作约束

推荐：一次主动开启，多次点击翻译

1. **用户在首页点击“开始屏幕翻译”。**
2. **完成悬浮窗权限引导与系统屏幕捕获同意。**Android 13+ 通知权限单独按系统版本说明和请求。
3. **获得捕获同意后启动 mediaProjection 前台服务，**在服务中建立捕获会话。
4. **用户切到游戏或阅读应用，**侧边显示浮键；主应用自身前台可隐藏浮键。
5. **同一活跃会话内多次点击浮键**取帧、框选、识别和翻译。
6. **用户结束翻译、系统停止投屏或会话失效时，释放资源并移除浮键；**重新开启需要新授权。

仅把应用退到后台，不能代替用户同意截屏。“开始屏幕翻译”可以是首次权限准备后的一个主按钮，但不能承诺首次打开就无提示截取其他应用。

> Android 14+ 要求每个捕获会话重新同意，授权结果及 MediaProjection 不能反复用于创建新的 VirtualDisplay；同一活跃会话可以取多帧。旋转或尺寸变化应更新当前显示 / Surface，不通过复用旧授权重新创建会话。Android 官方说明 (https://developer.android.com/about/versions/14/behavior-changes-14)
>
> 会话须有前台服务通知；Android 13+ 即使通知权限被拒绝，启动前台服务本身也不能省略通知。产品应说明系统显示位置可能不同，不能把授权拒绝和悬浮窗故障混为一谈。前台服务与通知权限 (https://developer.android.com/develop/ui/views/notifications/notification-permission#exemptions)
>
> Android 15 QPR1+ 有投屏状态栏入口，锁屏会结束投屏。必须正确响应系统停止回调，不能解锁后悄悄恢复旧捕获。MediaProjection (https://developer.android.com/media/grow/media-projection)

“轻便”与授权摩擦之间的取舍

模式

保留活跃投屏会话

使用感受

同一次使用中可反复点键框选。

代价

投屏持续存在，有系统提示且可能耗电，须测量。

首版决定

默认

模式

每次授权、取一张图后立即停止

使用感受

每次使用的资源边界直接。

代价

反复系统授权打断使用。

首版决定

后续按反馈评估

默认模式只有点键并框选后才做 OCR，不上传整屏图；但这不等于系统没有持续提供投屏画面，也不等于后台零耗电。ImageReader 的帧需及时消费 / 关闭，避免缓冲堆积；不在待命时做逐帧识别。

首版结束投屏就结束悬浮会话，不额外设计一个脱离投屏、永久驻留的浮键服务。

部分应用用 FLAG_SECURE 阻止截图，或禁止第三方悬浮层；此时可能黑屏或看不到浮键，应提示限制，不尝试绕过。截图冻结也不会暂停底下的游戏。安全窗口说明 (https://developer.android.com/security/fraud-prevention/activities)、隐藏悬浮窗 API (https://developer.android.com/reference/android/view/Window#setHideOverlayWindows(boolean))

易错点

投屏开着**不等于后台零耗电**；会话结束就移除浮键，不保留永久驻留服务。

本章要点

- 一次授权对应一个活跃会话；锁屏或系统停止后要重新开启。
- 默认保留活跃投屏，但必须测量耗电；结束投屏就结束悬浮会话。

4 PP-OCRv6 small 能否沿用

> **可以作为优先适配的同一套模型；还不能说 Android 端已跑通或达到了桌面精度。**官方提供 ONNX 文件，ONNX Runtime 官方支持 Android Java 接入，因此模型格式与运行时路线有依据。检测模型 (https://huggingface.co/PaddlePaddle/PP-OCRv6_small_det_onnx)、识别模型 (https://huggingface.co/PaddlePaddle/PP-OCRv6_small_rec_onnx)、ORT 移动部署 (https://onnxruntime.ai/docs/tutorials/mobile/)

当前工作区实测资产：

文件	实际大小
det.onnx	9,880,512 B
rec.onnx	21,159,378 B
两个模型合计	31,039,890 B，约 31.04 MB / 29.60 MiB
ppocrv6_dict.txt	74,947 B，另计

以上是模型文件，不是 APK 大小、安装占用或运行内存。现有桌面完整 OCR 目录约 199 MiB，包含 Python 等运行依赖，不应原封不动带到 Android。

可以复用：

模型、字典、官方配置、版本与哈希、截图样本、期望文字、翻译提示词和请求语义。

需要重新实现或适配：

Android 取图与颜色转换、tensor 前处理、DB 检测后处理、框扩张、透视裁剪、识别 resize / padding、CTC 解码、文字排序及模型生命周期。OpenCV 可承担轮廓和透视计算，DB 框扩张的具体库需在 OCR 原型中以一致性样本选定并锁版；不把它误当作 OpenCV 自动提供的完整 OCR。

对齐现有流水线的重点

1. 检测使用当前官方配置的颜色顺序、归一化和 DB 参数；先沿用最长边 1280，再按手机样本调节。
2. 识别先对齐现有 $[3, 48, 320]$ 配置及宽高比、padding 规则，不把所有文字行直接拉伸。
3. 字典顺序必须和模型输出一致；关闭未引入的方向分类和文档矫正。
4. 先对同一裁剪图比较桌面和 Android 输出，再讨论速度优化。

首版使用完整 ORT Android 包建立 CPU 基线。XNNPACK、NNAPI、量化和自定义精简 Runtime，只有基线显示收益需求时再逐项验证；不能承诺手机 GPU / NPU 一定更快。官方也建议先用预编译完整包开发，再按模型裁剪。ORT 移动部署 (https://onnxruntime.ai/docs/tutorials/mobile/)

模型卡的语种覆盖不等于本应用的验收范围。首版只对中英日建立证据；花体、竖排和极小字单独统计。若真实截图证明不达标，再做一次 ML Kit 对照实验，不预装双引擎或自动切换。ML Kit Android (https://developers.google.com/ml-kit/vision/text-recognition/v2/android)

易错点

模型文件 31 MB 不是 APK 体积；先让同一张裁剪图输出一致，再讨论速度。

本章要点

- 模型可以优先复用，但 Android 端的前后处理和输出一致性必须重做并验证。
- 先对齐同一裁剪图，再优化速度；语种覆盖不等于验收范围。

5 一次使用的数据流

开启会话 → 悬浮键 → 隐藏自己的浮层 → 等待干净的新帧 → 冻结 Bitmap → 手动矩形框选 → 裁剪区域 OCR → 显示可编辑原文 → 用户点击翻译 → 发送文字 → 逐步显示译文 → 关闭结果，恢复悬浮键

框选后允许调整边界，点击“识别”确认，再启动 OCR。截图展示坐标与原图像素坐标分开，统一换算缩放、留边、系统栏和旋转。识别使用原图裁剪，不使用为了显示而缩小的预览图。触发时取新帧，不从上一次选区遗留的截图取图。

首版优先整屏捕获，适合在不同应用间切换。系统若提供单应用捕获，要解释限制；选择单应用后，切换其他应用并不会自动改变捕获目标，应结束并重新选择。MediaProjection (https://developer.android.com/media/grow/media-projection)

最小职责划分

`MainActivity`
首页、设置、授权流程。

`CaptureSessionService`
投屏、通知、取帧、会话结束。

`overlay/`
浮键、冻结选框和译文卡片的显示与手势。

`ocr/`
模型加载及完整 OCR 流水线。

`translate/`
请求、流式解析、取消和会话缓存。

先做一个工程和一个完整闭环。外部配置、系统帧及网络响应在接收处校验一次，内部以明确类型传递；不建立层层重复校验。一次只处理当前操作，新操作取消旧工作；不为尚未出现的多任务场景扩展复杂调度器。

易错点

识别用**原图裁剪**，不用为了显示而缩小的预览图；每次触发都取新帧。

本章要点

- 取新帧、原图裁剪、先确认再识别；预览图不参与识别。
- 首版优先整屏捕获；单应用捕获要讲清目标不会随应用切换。

6 翻译、联网与数据

首版提供一个可编辑的服务配置：HTTPS 基础地址、模型标识、API Key、目标语言。用户自带 Key；应用不内置共享供应商密钥，也不需要自建账号后台。沿用现有兼容接口的语义，但不默认搬入桌面全部连接管理功能。

截图和 OCR 在设备内处理；只有用户确认翻译后的原文、目标语言和必要提示词发送到所选服务。首次设置说明这点；日志不记录截图、正文或密钥。服务提供者的数据处理方式取决于用户所选服务，不能宣称整条流程完全离线。

留在设备内

截图与识别

截图、OCR 和原文编辑在手机本地完成，不上传整屏图。

用户确认后发送

原文与目标语言

只有点击翻译后，原文、目标语言和必要提示词才会发送到所选服务。

不写入日志

截图、正文、密钥

日志不记录截图、正文或密钥；服务方如何处理取决于用户选择的服务。

Key 用 Android Keystore 生成的密钥加密后落盘；普通配置可放 DataStore，二者分开。Keystore 官方文档 (https://developer.android.com/privacy-and-security/keystore)

会话缓存的键包括原文、目标语言和影响结果的服务 / 模型 / 提示词配置。修改原文清除旧译文，只缓存正常完成的结果。断网或额度问题保留可编辑原文，用户能重试；不自动反复发送收费请求。

手机上的 127. 0. 0. 1 指手机自身，不能照搬桌面 LM Studio / Ollama 地址。连接电脑局域网服务是后续独立功能，需处理网络、安全策略和地址，不包含在首版。

易错点

设备内识别**不等于整条流程离线**；翻译请求会发送原文。

本章要点

- 用户自带 Key；识别留在设备内，只有确认翻译后才发送原文。
- 缓存与“修改原文清空旧译文”的语义要一致；手机上的 127.0.0.1 是手机自己。

7 通用版兼容与发布基线

- **最低系统：**建议最低 Android 9 / API 28。这是覆盖面与适配成本的产品取舍，不是截图 API 的硬性最低要求；以所选 ORT / OpenCV 版本实际要求复核。
- **主要 ABI：**主要发布 `arm64-v8a`。32 位旧机、x86 平板不在首批承诺范围，模拟器包另计；“通用”主要指不依赖特定品牌和 Google 服务。
- **目标 SDK：**`compileSdk` / `targetSdk` 至少按 API 36 规划，依赖选开发时稳定兼容版本并锁定。按当前官方页面，2026-08-31 起 Google Play 普通新应用及更新要求 target Android 16 / API 36。官方要求 (https://developer.android.com/google/play/requirements/target-sdk)
- **首轮交付：**首轮以可安装的签名 APK 试用，后续商店发布另准备 AAB、说明和审核资料。
- **原生库：**ORT、OpenCV 含原生库，必须检查所有 `.so` 的 ABI 与 16 KB page size 支持，并在对应环境实际启动。模型能加载不等于整包兼容。Android 16 KB 指南 (https://developer.android.com/guide/practices/page-sizes)
- **体积与许可：**保留模型与实际依赖许可证；不把“用了原生栈”当作小体积证明。先统计体积构成，明显超预算后再裁剪依赖。

> **“通用”不等于所有手机、所有应用都能用。**首批以 Android 9+ ARM64 手机为范围，不依赖特定品牌或 Google 服务。

易错点

模型能加载**不等于整包兼容**；要检查 ABI、16 KB page size，并在对应环境实际启动。

本章要点

- 最低 Android 9、主要 ARM64、按 API 36 规划；2026-08-31 起商店要求 target API 36。
- 原生库要查 ABI 与 16 KB page size；体积与许可都要单独核对。

8 目前证据与待验证项

✓ 已核对：

现有 STOR 模型及流水线、模型文件大小、官方 ONNX 与 Android Runtime 路线、系统授权 / 前台服务 / 悬浮窗约束。

? 尚未验证：

Android 模型加载、DB 后处理等输出一致性、真实取帧时序、机型耗时 / 内存 / 耗电、OEM 后台行为、最终 APK 体积。

所有性能数字应来自后续原型，不从桌面推断。下一步与验收见开发计划，面向产品决策的解释见产品经理讲解书。

易错点

桌面数据不能直接当手机结论；**性能数字必须来自后续原型**。

本章要点

- 已核对的是模型、路线与系统约束；未验证的是真机输出、时序、性能与体积。
- 不要把桌面结果或“能加载”当作手机端体验结论。

配套文档

产品经理讲解书 | 开发计划与验收 | 技术栈方案原文（Markdown）

内容整理自《Android 屏幕翻译：技术栈方案》（2026-10-27 版）。本页为学习版，术语提示、章节要点与自测为辅助；事实、边界与状态以原文为准。

STOR Android 方案，版本 2026-10-27